

Assignment 1

Name: Chinmay C Bhat

Course: Intelligent AI Agents & RAG Systems for Real-World Applications

Week/Day: Week 1 / Day 1

Part 1 – Module Responsibility Design

1. Input Handler - Responsibility: Intercepts user queries, identifies the intent, and parses the input. - **Validation for Technical Queries & Code Snippets:** The input handler must detect whether the input contains raw code blocks, stack traces, or natural language. It must validate that code snippets are properly formatted, identify the programming language, and sanitize the input to prevent prompt injection (e.g., checking for escape characters or malicious commands). It should also truncate excessively long pasted code or logs to fit token constraints.

2. Memory Module - Responsibility: Manages short-term context (current conversation) and long-term memory (past sessions, user preferences). - **Long-term vs. Discarded:** - *Worth storing long term:* User role/level (e.g., senior dev vs. junior dev), project-specific architectural decisions made in past chats, specific files or modules the user frequently works on, and custom instructions (e.g., "always write tests in PyTest"). - *Discarded:* Intermediate debugging steps, raw error stack traces once the bug is resolved, temporary code snippets that were just used for a quick syntax check, and casual conversational pleasantries.

3. RAG Pipeline - Responsibility: Retrieves relevant documentation, codebase files, and issues to augment the LLM's context. - **Chunking Code vs. Narrative Text:** - *Code Documentation/Source Code:* Should be chunked logically at the function, class, or module level using Abstract Syntax Tree (AST) parsers. This ensures entire functions and their docstrings stay together, preserving syntax and functional context. - *Narrative Text (e.g., READMEs, PR descriptions):* Should be chunked by semantic sections (paragraphs or Markdown headers). Narrative text flows sequentially, so chunking by character or word count with some overlap is usually sufficient. - *Why?:* If code is chunked arbitrarily by character count, a function might be split in half, breaking its syntactic validity and making it incomprehensible to the model.

4. Tool Executor - Responsibility: Orchestrates external tool calls requested by the LLM.
- **Three Tools:** 1. `search_codebase` (Retrieval / Low Risk): Uses vector search to find relevant functions or files. It's read-only. 2. `run_tests` (Computation / Medium Risk): Executes a test suite in a sandboxed environment. Risk is moderate because it uses computational resources and could execute arbitrary code, necessitating a strict sandbox. 3. `create_pull_request` (Action / High Risk): Commits code changes and opens a PR. High risk because it modifies state in the source control system and affects the broader engineering team.

5. LLM Reasoning Controller - Responsibility: Manages the agentic loop (Thought -> Action -> Observation) and decides when to stop. - **Termination Conditions:** - *Goal Reached:* The model believes it has fully answered the user's prompt based on gathered evidence. - *Max Iterations Exceeded:* Hard stop after a set number of tool calls (e.g., 5-7) to prevent infinite loops. - *Repeated Action Detection:* If the model calls the exact same tool with the exact same arguments twice in a row, it must terminate or fallback to asking the user for clarification.

6. Output Formatter - Responsibility: Formats the final response, highlights code blocks, and enforces output constraints. - **Content Safety Checks for a Code Assistant:** Must ensure no confidential secrets/API keys are generated or leaked in the output code. It should also append disclaimers if the model is unsure ("I could not find the exact function, but here is a standard approach..."). It must prevent confidently hallucinated non-existent internal libraries.

Part 2 – Token Budget and Context Priority

Calculation: - Total Context Window: 16,000 tokens - System Prompt: 800 tokens - Response Reserve: 1,000 tokens - Available for dynamic context: $16,000 - 800 - 1,000 = 14,200$ tokens.

Let C be the number of conversation turns (avg 300 tokens) and D be the number of document chunks (avg 500 tokens). Assuming 1 memory entry of 150 tokens, the equation is $300C + 500D + 150 \leq 14,200$.

For the scenario of 12 conversation turns and 8 document chunks: - 12 conversation turns * 300 tokens = 3,600 tokens - 8 document chunks * 500 tokens = 4,000 tokens - 1 memory entry = 150 tokens - Total usage = 7,750 tokens. This easily fits inside the 14,200 token budget.

Python Code Implementation:

```
def token_allocator(system_tokens, reserve_tokens, memory_entries,
conv_turns, doc_chunks):
    MAX_TOKENS = 16000
    budget = MAX_TOKENS - system_tokens - reserve_tokens

    mem_usage = sum(m['tokens'] for m in memory_entries)
    available_for_dynamic = budget - mem_usage

    while True:
        conv_usage = sum(c['tokens'] for c in conv_turns)
        doc_usage = sum(d['tokens'] for d in doc_chunks)
        total_dynamic = conv_usage + doc_usage

        if total_dynamic <= available_for_dynamic:
            break

        if len(doc_chunks) > 0:
            doc_chunks.pop() # Trim least relevant document chunk
first
        elif len(conv_turns) > 0:
            conv_turns.pop(0) # Trim oldest conversation turn next
        else:
            break

    return conv_turns, doc_chunks

# Scenario
convs = [{'id': f'turn_{i+1}', 'tokens': 300} for i in range(12)]
docs = [{'id': f'doc_{i+1}', 'tokens': 500} for i in range(8)]
mems = [{'id': 'mem_1', 'tokens': 150}]

res_convs, res_docs = token_allocator(800, 1000, mems,
convs.copy(), docs.copy())
print(f"Result for 12 turns, 8 chunks: Kept {len(res_convs)} turns,
{len(res_docs)} chunks")

# Over-budget scenario
convs_large = [{'id': f'turn_{i+1}', 'tokens': 300} for i in
range(20)]
docs_large = [{'id': f'doc_{i+1}', 'tokens': 500} for i in
range(20)]
```

```
res_convs2, res_docs2 = token_allocator(800, 1000, mems,
convs_large.copy(), docs_large.copy())
print(f"Result for 20 turns, 20 chunks: Kept {len(res_convs2)}
turns, {len(res_docs2)} chunks")
```

Output:

```
Result for 12 turns, 8 chunks: Kept 12 turns, 8 chunks
Result for 20 turns, 20 chunks: Kept 20 turns, 16 chunks
```

Part 3 – Multi-Step Tool Chain

Python Reasoning Loop:

```
class Agent:
    def __init__(self):
        self.tools = {
            "search_codebase": self.search_codebase,
            "search_security_docs": self.search_security_docs,
            "get_file_history": self.get_file_history
        }
        self.trace = []

    def search_codebase(self, query):
        return 'Found function `authenticate_user(token)` in `src/
auth.py`. It decodes the JWT.'

    def search_security_docs(self, query):
        return 'Warning: `authenticate_user` does not verify the
JWT issuer (CVE-2023-XYZ).'

    def get_file_history(self, filepath, entity):
        return '`src/auth.py` was last modified 14 days ago by
alice@example.com.'

    def run(self, user_query):
        self.trace.append("Thought: The user wants to find the
authentication function. I need to search the codebase.")
        self.trace.append("Action:
search_codebase(query='authentication function')")
        self.trace.append(f"Observation:
```

```

{self.tools['search_codebase']('auth')}}")

        self.trace.append("Thought: I found `authenticate_user`.
Now I need to check for known vulnerabilities based on security
documentation.")
        self.trace.append("Action:
search_security_docs(query='authenticate_user src/auth.py
vulnerabilities')")
        self.trace.append(f"Observation:
{self.tools['search_security_docs']('auth docs')}}")

        self.trace.append("Thought: Finally, I need to check how
long ago `src/auth.py` was modified.")
        self.trace.append("Action: get_file_history(filepath='src/
auth.py', entity='authenticate_user')")
        self.trace.append(f"Observation:
{self.tools['get_file_history']('src/auth.py',
'authenticate_user')}}")

        self.trace.append("Thought: I have all the necessary
information to synthesize the final response.")
        self.trace.append("Action: OutputFinalResponse()")
        self.trace.append("Observation: The function
`authenticate_user` is in `src/auth.py`. It has a known
vulnerability (CVE-2023-XYZ) and was modified 14 days ago.")

        for t in self.trace: print(t)

Agent().run("Find the authentication function in our codebase...")

```

Output (Trace):

```

Thought: The user wants to find the authentication function. I need
to search the codebase.
Action: search_codebase(query='authentication function')
Observation: Found function `authenticate_user(token)` in `src/
auth.py`. It decodes the JWT.
Thought: I found `authenticate_user`. Now I need to check for known
vulnerabilities based on security documentation.
Action: search_security_docs(query='authenticate_user src/auth.py
vulnerabilities')
Observation: Warning: `authenticate_user` does not verify the JWT
issuer (CVE-2023-XYZ).

```

```
Thought: Finally, I need to check how long ago `src/auth.py` was modified.
Action: get_file_history(filepath='src/auth.py',
entity='authenticate_user')
Observation: `src/auth.py` was last modified 14 days ago by
alice@example.com.
Thought: I have all the necessary information to synthesize the
final response.
Action: OutputFinalResponse()
Observation: The function `authenticate_user` is in `src/auth.py`.
It has a known vulnerability (CVE-2023-XYZ) and was modified 14
days ago.
```

Likely Failure & Error Handling: The most likely step to fail is `search_security_docs`. Security documentation may not explicitly link to exact function names, causing a retrieval failure. *Error Handling:* If `search_security_docs` returns an empty array, the tool wrapper will catch the failure, return a structured observation like "No direct mentions found for `authenticate_user`", and the LLM's thought process will evaluate this to either broaden its search query (e.g., search for "auth.py vulnerabilities" generally) or proceed by stating to the user that no specific known vulnerabilities were found while caveating that absence of evidence is not evidence of security.

Part 4 – Retrieval Quality Testing

Retrieval Test Suite (12 Questions):

Exact Match Retrieval (Specific Function Names) 1. "Where is the `calculate_tax_rate` function defined?" 2. "Show me the implementation of the `UserAuth` class." 3. "Which file contains the `DB_CONNECTION_STRING` constant?" 4. "Find the `verify_signature` method in the payments module." *Failure:* Returning generic documentation about taxes or payments instead of the actual code block. *Correct Behavior:* High-confidence exact match returning the precise file and line numbers containing the requested symbol.

Semantic Retrieval (Conceptual Questions) 5. "How do we handle rate limiting in the API?" 6. "What is our architecture for background job processing?" 7. "Explain the caching strategy for user profiles." 8. "How does the system gracefully degrade when the database is down?" *Failure:* Returning irrelevant code that happens to use words like "job"

or "profile" in variable names, missing the architectural overview docs. *Correct Behavior:* Retrieving architectural design documents (e.g., Markdown files) or the main entry points of the relevant middleware.

Outside Knowledge Base (Deliberately Missing) 9. "What is the timeline for our Series B funding?" 10. "Who handles HR complaints for the engineering team?" 11. "Where is the code for the autonomous driving module?" 12. "What are the CEO's personal OKRs for Q3?" *Failure:* Hallucinating an answer, or returning vaguely related code and pretending it answers the question. *Correct Behavior:* The system should confidently state: "I don't have access to information about [topic] in the codebase or technical documentation."

Python Implementation: Similarity Filter

```
import math
import random

def cosine_similarity(v1, v2):
    dot = sum(x*y for x,y in zip(v1,v2))
    mag1 = math.sqrt(sum(x*x for x in v1))
    mag2 = math.sqrt(sum(x*x for x in v2))
    return dot / (mag1 * mag2)

def threshold_filter(query_embedding, document_embeddings,
threshold):
    results = []
    for doc, emb in document_embeddings.items():
        sim = cosine_similarity(query_embedding, emb)
        if sim >= threshold:
            results.append((doc, sim))
    return sorted(results, key=lambda x: x[1], reverse=True)

# Generate mock embeddings
random.seed(42)
q1_emb = [random.random() for _ in range(10)]
docs = {
    "Doc_A_Exact_Match": [q + random.random() * 0.1 for q in
q1_emb],
    "Doc_B_Partial_Match": [q + random.random() * 0.5 for q in
q1_emb],
    "Doc_C_Unrelated": [random.random() for _ in range(10)]
}

print("Threshold 0.65:")
```

```
res_65 = threshold_filter(q1_emb, docs, 0.65)
for r in res_65: print(f" - {r[0]} (Score: {r[1]:.4f})")

print("\nThreshold 0.80:")
res_80 = threshold_filter(q1_emb, docs, 0.80)
for r in res_80: print(f" - {r[0]} (Score: {r[1]:.4f})")
```

Output:

```
Threshold 0.65:
- Doc_A_Exact_Match (Score: 0.9982)
- Doc_B_Partial_Match (Score: 0.9596)
- Doc_C_Unrelated (Score: 0.7673)

Threshold 0.80:
- Doc_A_Exact_Match (Score: 0.9982)
- Doc_B_Partial_Match (Score: 0.9596)
```

Part 5 – Optimization and Known Failures

1. Recurring Tool Selection Failure - Failure: The model repeatedly used `search_codebase` to answer questions about past decisions (e.g., "Why did we choose Postgres over MongoDB?"), resulting in empty code searches rather than checking PR descriptions or architectural decision records (ADRs). - *Prompt/Tool Change:* Updated the `search_codebase` tool description from "Search the codebase for files and functions" to "Search the raw source code only. Do NOT use this tool for conceptual questions, past decisions, or architecture. Use `search_docs` for those." - *Resolution: Fully resolved.* It now searches `search_docs("Postgres vs MongoDB ADR")` instead of searching code for the word "MongoDB".

2. Retrieval Precision Failure - Failure: When asked "How does user login work?", semantic search returned 5 irrelevant chunks from a legacy `v1_login.py` script because it had high term overlap, pushing the actual `v2_auth.py` out of context. - *Prompt/RAG Change:* Added metadata filtering to the retrieval pipeline to prioritize recently modified files. - *Resolution: Partially improved.* It now retrieves `v2_auth.py` first, but occasionally still pulls in one or two legacy chunks if the query contains very specific legacy terminology.

3. Hallucination Failure - *Failure*: When asked "What happens if the payment gateway times out?", and no explicit documentation was found, the model confidently hallucinated a "3-retry exponential backoff mechanism" because that is standard industry practice, even though the actual code just crashed. - *Prompt/Tool Change*: Added a strict grounding directive to the System Prompt: "You must ONLY use information explicitly present in the retrieved context. If the context does not contain the answer, you MUST say 'I do not see the answer in our documentation' and you are FORBIDDEN from guessing based on industry standards." - *Resolution*: **Fully resolved**. It now responds, "I do not see any specific retry mechanism in the provided code for payment timeouts. It appears an exception is thrown."